my-agents-cli-mod

A lab project generated with `agents-cli create my-agents-cli-mod` (agents-cli **1.13.0**, template `adk_python`), in which `app/agent.py` was modified into a **general-purpose AI assistant that reports real-time local time and weather for cities worldwide and also answers general questions**.

> Korean version: README-KR.md · Template default guide (English summary): README.md

---

1. Changes from the Original Template (`app/agent.py`)

Item	Details
Agent name	`root_agent.name = "my_agents_cli_mod"` (kept identical to `root_agent_name` in `agents-cli-manifest.yaml`), `App(name="app")`
Model	`ADK_MODEL` environment variable (default `gemini-3.8-flash`), retried with `HttpRetryOptions(attempts=3)`
Tools	`geocode` · `get_current_time` · `get_weather` (3 tools)
External API	Open-Meteo Geocoding / Forecast API — **no API key required**
Korean support	Korean → English city name dictionary (`KOREAN_CITY_MAP` , 28 cities incl. Seoul, Busan, Tokyo, New York), Korean mapping of WMO weather codes (`WMO_WEATHER_CODES`)
Offline fallback	On network failure or when `AGENT_OFFLINE=1` , uses backup coordinates/timezones for major cities (`_OFFLINE`) and a default weather response (`source: offline_fallback`)
Instruction	Time/weather questions are answered in Korean using tools; all other general questions (trivia, programming, translation, etc.) are answered directly without tools

Tool Summary

Tool	Input	Returns
`geocode(location)`	City/region name (Korean or English)	`display_name`, `latitude`, `longitude`, `timezone`
`get_current_time(location, query, latitude, longitude)`	City name or coordinates	`%Y-%m-%d %H:%M:%S %Z (UTC%z)` ,
`get_weather(location, query, latitude, longitude)`	City name or coordinates	Weather condition, temperature, feels-like temperature, humidity, wind speed

---

2. Environment Variables (`.env`)

`app/agent.py` reads `.env` from two locations:

1. `my-agents-cli-mod/.env` (takes precedence if present, `override=True`)
2. The lab root `ADK_Labs/.env` (`_ROOT.parent.parent / ".env"`)

Therefore, **configuring only the root `ADK_Labs/.env` is enough** for this project to work. This folder has no separate `.env` .

```
# ① Google AI Studio
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=<your-gemini-api-key>

# ② Vertex AI (requires gcloud auth application-default login)
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=<your-gcp-project-id>
# GOOGLE_CLOUD_LOCATION=global

# (Optional) Change model / offline mode
# ADK_MODEL=gemini-3.5-flash-lite
# AGENT_OFFLINE=1
```

- When `GOOGLE_GENAI_USE_VERTEXAI=TRUE`, `GOOGLE_API_KEY` and `GEMINI_API_KEY` are removed and `GOOGLE_CLOUD_LOCATION` defaults to `global`.
- The template-generated `.env.example` uses the `GOOGLE_GENAI_USE_ENTERPRISE` / `GEMINI_API_KEY` format. To use that format, copy it to `.env` in this folder.

---

3. Requirements

- **uv**: Python package manager — Installation Guide
- **agents-cli**: `uv tool install google-agents-cli` (also installed together with the root `requirements.txt`)
- **Google Cloud SDK** (for deployment): Installation Guide
- `a2a-sdk[http-server]>=1.0,<2` `pyproject.toml`

---

4. How to Run

Run from the `part03_cli/my-agents-cli-mod` directory:

```
cd part03_cli/my-agents-cli-mod

# 1. Install dependencies (creates .venv / uv.lock)
agents-cli install

# 2. Launch the local playground web UI
agents-cli playground

# 3. Run the FastAPI server directly (web UI + REST/SSE + A2A)
uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000 --reload

# 4. Run with the ADK CLI (agent folder = app)
uv run adk web --port 8008
uv run adk run app

# 5. Offline mode (uses backup data without calling the external weather API) - PowerShell
$env:AGENT_OFFLINE="1"; agents-cli playground
```

Example prompts:

- `What's the feels-like temperature in Paris?`
- `Explain Python list comprehensions` (answered without calling tools)

---

5. Testing · Evaluation · Deployment

Command	Description
`agents-cli lint`	Code quality checks (ruff · ty · codespell)
`uv run pytest tests/unit`	Unit tests (`test_dummy.py` , no LLM required)
`uv run pytest tests/integration`	Integration tests (`test_agent.py` : Runner streaming, `test_server_e2e.py` : FastAPI & A2A E2E) — requires a Gemini key
`agents-cli eval run`	Runs `tests/eval/datasets/basic-dataset.json` (3 cases: greeting, San Francisco weather, capital of France) and scores with `custom_response_quality` (LLM-as-judge, 1–5 points)
`agents-cli deploy`	`deployment_target: agent_runtime` , `us-east1` , `deployment_metadata.json`
`agents-cli publish gemini`	Registers the deployed agent with Gemini
`agents-cli scaffold enhance`	Adds CI/CD and Terraform code (this project's `deployment/terraform/` was generated by this command)
`agents-cli infra cicd`	One-click CI/CD pipeline and infrastructure setup
`agents-cli scaffold upgrade`	Upgrades the template while preserving user customizations

```
gcloud config set project <your-project-id>
agents-cli deploy --no-confirm-project
```

```
                                                    COPY ./pyproject.toml ./README.md
./uv.lock* ./ ,.            README.md
```

---

6. Server Architecture (`app/fast_api_app.py`)

- `get_fast_api_app(agents_dir=..., web=True, ...)` creates the ADK web UI and default API routes such as `/run` and `/run_sse`.
- In the lifespan hook, it creates `Runner(app=adk_app, auto_create_session=True)` and attaches the **A2A JSON-RPC endpoint** `/a2a/app` via `attach_a2a_routes()` (`is_a2a: true`).
- `attach_reasoning_engine_routes(app)` adds `reasoning_engine` proxy routes for the Cloud console playground.
- Sessions are `in_memory` (manifest `session_type`); telemetry is exported to Cloud Trace and others when `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true` .
- Uses port `8000` when run directly, and port `8080` in the container (Dockerfile).

Use the A2A Inspector to test A2A interoperability.

---

7. Supplementary Material: ADK Web Architecture Slides

File	Description
`generate_adk_web_ppt.py`	Script that generates ADK Web architecture slides (8 slides) using `python-pptx`
`ADK_Web_Architecture_Guide.pptx`	Output of the script above — overall architecture, web UI packaging & FastAPI mounting, server startup lifecycle, `/run_sse` streaming, developer tools, `adk web` vs `adk api_server` comparison, run options summary

```
uv pip install python-pptx
uv run python generate_adk_web_ppt.py                  # Default output:
ADK_Web_Architecture_Guide.pptx
uv run python generate_adk_web_ppt.py my_slides.pptx   # Specify output file name
```

---

8. 📁 Detailed Directory Structure and Creation Timing

```
my-agents-cli-mod/
├── app/                                # 🟢 Core agent application (agent_directory)
│   ├── __init__.py                     # 🟢 Python package declaration
│   ├── agent.py                        # 🔨 Real-time time/weather tools + general assistant
(modified)
│   ├── fast_api_app.py                 # 🟢 FastAPI server: web UI + REST/SSE + A2A
│   ├── app_utils/                      # 🟢 Helper utilities
│   │   ├── a2a.py                      # 🟢 A2A route wiring and Agent Card handling
│   │   ├── reasoning_engine_adapter.py # 🟢 reasoning_engine proxy for the Cloud console
playground
│   │   └── services.py                 # 🟢 Session & artifact service factory
│   ├── .adk/                           # ⚡ [Runtime] Created on local execution
│   │   └── session.db                  # ⚡ SQLite DB for session & conversation state
│   └── __pycache__/                    # ⚡ [Compile] Bytecode cache
│
├── deployment/terraform/               # ⚡ [scaffold enhance] Terraform code for GCP
infrastructure
│   ├── shared/                         # ⚡ completions.sql, genai_logs_schema.json,
dummy_source.b64
│   └── single-project/                 # ⚡ apis · iam · service · storage · telemetry .tf,
vars/env.tfvars
│
├── tests/                              # 🟢 Tests and evaluation
│   ├── unit/test_dummy.py              # 🟢 Basic unit test
│   ├── integration/
│   │   ├── test_agent.py               # 🟢 Runner SSE streaming integration test
│   │   └── test_server_e2e.py          # 🟢 FastAPI & A2A endpoint E2E test
│   └── eval/
│       ├── eval_config.yaml            # 🟢 Evaluation metric settings (custom_response_quality,
agent_turn_count)
│       ├── response_quality.py         # 🟢 LLM-as-judge response quality scorer
│       └── datasets/
│           ├── basic-dataset.json      # 🟢 3 evaluation queries
│           └── README.md               # 🟢 Evaluation dataset usage
│
├── .venv/                              # ⚡ [install] Virtual environment from agents-cli
install / uv sync
├── uv.lock                             # ⚡ [install] Package version lockfile
├── deployment_metadata.json            # ⚡ [deploy] Deployed Agent Runtime (reasoningEngines)
ID & timestamp
│
├── agents-cli-manifest.yaml            # 🟢 agents-cli metadata (template, agent name, region,
deploy target)
├── pyproject.toml                      # 🟢 Dependencies and ruff/ty/pytest settings
├── Dockerfile                          # 🟢 Container build spec (python:3.12-slim, port 8080)
├── .env.example                        # 🟢 Environment variable template
├── .gitignore / .gcloudignore          # 🟢 Git / GCP upload ignore rules
├── GEMINI.md                           # 🟢 Development guide for coding agents
(Antigravity/Gemini)
├── README.md                           # 🔨 Project summary (English, required for the
Dockerfile build)
│
├── generate_adk_web_ppt.py             # 🔨 [User added] ADK Web architecture slide generator
script
├── ADK_Web_Architecture_Guide.pptx     # 🔨 [User added] Slides generated by the script above
├── README-KR.md / README-KR.pdf        # 🔨 [User added] Detailed Korean guide
└── README-EN.md / README-EN.pdf        # 🔨 [User added] Detailed English guide (this file)
```

Icon	Category	Description	Representative Files
🟢	**Initial Scaffold**	Auto-generated from the template by `agents-cli create`	`app/fast_api_app.py`, `tests/`, `Dockerfile`, `agents-cli-manifest.yaml`
⚡	**Dynamically Generated**	Created afterwards by `scaffold enhance deploy` running · `pytest`	`.venv/`, `uv.lock`, `deployment/`, `deployment_metadata.json`, `app/.adk/session.db`
✏️	**User Modified/Added**	Files edited from the template or added by the developer	`app/agent.py`, `README*.md`, `generate_adk_web_ppt.py`, `ADK_Web_Architecture_Guide.pptx`